

Тема 5. 1. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Съдържание:

1. Оператори и макроси.

1.1. Оператори

1.1.1. Аритметични оператори

1.1.2. Сравнителни оператори

1.1.3. Логически оператори

1.1.4. Побитови оператори

2. Макроси

2.1. Разбиране на времето на изпълнение

2.2. Модули и функции по време на изпълнение

2.3. Чисти модули Erlang

2.4. Динамично извикване на функции

2.5. Стартиране на времето за изпълнение

2.6. Интерактивна обвивка

2.7. Изпълнение на скриптове

2.8. Инструментът за смесване

2.9. Макроси

2.9.1. Създаване на първият макрос

2.9.2. Макро хигиена

2.9.3. Околната среда

2.9.4. Частни макроси

3. Създаване на потребителски функции.

3.1. Обръщение към функция.

3.2. Вградени функции.

3.3. Function type annotations (Анотации за тип функция)

3.4. (Function heads) Функционални глави

3.5. Function overloading (Претоварване на функциите)

3.6. Referencing functions (Справочни функции)

3.7. Calling anonymous functions (Извикване на анонимни функции)

3.8. Labelled arguments (Обозначени аргументи)

3.9. Анонимни функции

3.10. Capture Operator & (Оператор на улавяне &)

3.11. Multiple Implementations (Множество изпълнения)

3.12. Lexical Scope (Лексикален обхват)

3.13. Named Functions (Наименувани функции)

3.14. Modules (Определяне на модули)

3.15. In-Line Definition (Вградена дефиниция)

3.16. Function Guards (Функционални зщити)

3.17. Персонализирана защита

3.18. Multi-Clause Functions (Функции с много клаузи)

3.19. Аргументи за съвпадение на шаблони

3.20. Частни функции

3.21. Аргументи по подразбиране

3.22. Стенографията &

3.23. Съвпадащ модел

3.24. Наименувавани функции

3.25. Именуване на функции и Arity

3.26. Функции и съвпадение на шаблони

3.27. Частни функции

3.28. Защитници

3.29. Аргументи по подразбиране

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

1. Оператори и макроси.

1.1. Оператори

Операторът е символ, който казва на компилатора да извърши определени математически или логически манипулации. Има много оператори, осигурени от Еликсир. Те са разделени в следните категории:

- Аритметични оператори;
- Сравнителни оператори;
- Булеви оператори.

1.1.1. Аритметични оператори

Следващата таблица изброява всички аритметични оператори, поддържани от езика Elixir. Ако променливата **A** има стойност 10, а променливата **B** има стойност 20, то:

Таблица 1.

Оператор	Описание	Пример
+	Добавя 2 числа.	A + B ще даде 30
-	Изважда второто число от първото.	A - B ще даде -10
*	Умножава две числа.	A * B ще даде 200
/	Разделя първото число на второто. Дава резултат число с плаваща запетая	A / B ще даде 0,5.
DIV (division)	Тази функция се използва за получаване на цялата част от деление на двете числа.	div (10, 20) ще даде 0
REM (remainder)	Тази функция се използва за получаване на остатък от деление на две числа.	rem (A, B) ще даде 10

```
div 10, 2 # 5  
div 10, 3 # 3  
div 10, 4 # 2  
div 10, 5 # 2
```

```
rem 10, 2 # 0  
rem 10, 3 # 1  
rem 10, 4 # 2  
rem 10, 5 # 0
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

1.1.2. Сравнителни оператори

Сравнителните оператори в Elixir най-често се срещат и използват подобно на повечето други езици. Следващата Таблица 2 обобщава операторите за сравнение в Elixir. Нека променлива **A** съдържа 10 и променлива **B** съдържа 20, тогава:

Таблица 2.

Оператор	Описание	Пример
==	Проверява дали стойността вляво е равна на стойността вдясно (типът нулира стойностите, ако не са от същия тип).	A == B ще даде false
!=	Проверява дали стойността отляво не е равна на стойността отдясно.	A != B ще даде true
===	Проверява дали типът на стойността отляво е равен на типа на стойността отдясно , ако да, тогава проверете същото за стойността.	A === B ще даде false
!==	Същото като по-горе, но тестове за неравенство вместо равенство .	A !== B ще даде true
>	Проверява дали стойността на левия операнд е по-голяма от стойността на десния операнд; ако да, тогава условието става вярно.	A > B ще даде false
<	Проверява дали стойността на левия операнд е по-малка от стойността на десния операнд; ако е така, състоянието става вярно.	A < B ще даде true
>=	Проверява дали стойността на левия операнд е по-голяма или равна на стойността на десния операнд; ако е така, състоянието става вярно.	A >= B ще даде false
<=	Проверява дали стойността на левия операнд е по-малка или равна на стойността на десния операнд; ако да, тогава условието става вярно.	A <= B ще даде true

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

1.1.3. Логически оператори

Elixir предоставя 6 логически оператора: и, или, не, &&, || както и !. Първите три са строги булеви оператори, което означава, че очакват първият им аргумент да бъде булев. Небулеви аргумент ще изведе грешка. Докато следващите три, &&, || както и ! не са строги, не изискват от нас да имаме първата стойност строго, булева стойност. Те работят точно като техните строги алтернативи. Да предположим, че променлива A е true и променлива B съдържа 20, тогава

Таблица 3.

Оператор	Описание	Пример
and	Проверява дали и двете предоставени стойности са правилни, ако да, тогава връща стойността на втората променлива. (Logical and)	A and B , ще дадат 20
or	Проверява дали някоя от посочените стойности е вярна. Връща коя стойност е вярна. В началото връща false. (Logical or)	A or B , ще даде True
not	Унарен оператор, който инвертира стойността на посочения вход.	not A , ще даде False
&&	Не е строг и работи по същия начин and , но не очаква, че първият аргумент е логичен.	B && A , ще даде 20
 	Работи по същия начин като or , но не очаква първият аргумент да е булев.	B A , ще даде True
!	Не строго не. Работи по същия начин, както не, но не очаква аргументът да е булев.	!A , ще даде False

ЗАБЕЛЕЖКА - и, или, && и || са оператори на късо съединение. Това означава, че ако първият аргумент е и е неверен, то вече няма да се проверява втория. И ако първият аргумент е или е истина, тогава няма да се провери втория. Например: **false and raise("An error")**

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

1.1.4. Побитови оператори

Побитовите оператори работят върху битове и изпълняват побитови операции. Elixir предоставя битови модули, като част от битовия пакет, така че трябва да се използва битов модул за да ги използвате. За да се използва, трябва да се въведе следната команда в шела - **use Bitwise**.

Нека $A = 5$ и $B = 6$:

Таблица 4.

оператор	Описание	Пример
&&&	Побитово and операторът копира малко, за да го прехвърли, ако съществува и в двата операнда.	A &&& B , ще даде 4
 	Побитово or операторът копира малко, за да получи резултата, ако той съществува в някой от операндите.	A B , ще даде 7
>>>	Побитовият десен оператор на смяна измества първия бит на операнда надясно с номера, посочен във втория операнд.	A >>> B , ще даде 0
<<<	Побитовият ляв оператор за смяна измества първите битове на операнда наляво с номера, посочен във втория операнд.	A <<< B , ще даде 320
^^^	Побитовият XOR оператор копира малко, за да получи резултата само ако е различен и за двата операнда.	A ^^^ B , ще даде 3
~~~	Унарният побитов оператор не обръща битове с посочения номер.	~~~ A ще даде -6

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

II. Макроси в Elixir

Макросите в Elixir предлагат мощен инструмент за разширяване на езика. Те позволяват на разработчиците да генерират код динамично по време на компилация, което значително увеличава гъвкавостта и производителността на кода.

Създаване на макроси:

Макросите се дефинират с ключовата дума `defmacro`. Те могат да приемат аргументи и да генерират произволен код, който ще бъде компилиран в последствие.

Пример:

```
defmodule MyMacros do  
  defmacro say_hello(name) do  
    quote do  
      IO.puts("Hello, #{unquote(name)}!")  
    end  
  end  
end
```



```
require MyMacros  
MyMacros.say_hello("Elixir") # Изход: Hello, Elixir!
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Подробно описание

1. Дефиниране на модул

```
defmodule MyMacros do
```

Това създава нов модул с име MyMacros. В Elixir модулите групират функции и макроси, и служат за организационна структура на кода.

2. Дефиниране на макрос

```
defmacro say_hello(name) do
```

Тази линия дефинира макрос с име say_hello, който приема един аргумент – name. Думата defmacro обозначава, че става въпрос за макрос, а не просто функция. Макросите в Elixir се използват, за да преобразуват код по време на компилация, а не по време на изпълнение.

3. quote в кода

```
quote do
```

```
IO.puts("Hello, #{unquote(name)}!")
```

```
end
```

Вътре в макроса, quote е специална конструкция, която задава блок от код, който трябва да бъде компилиран и изпълнен. Кода, който е в quote блока, ще бъде генериран вместо самия макрос, когато той бъде извикан.

```
IO.puts("Hello, #{unquote(name)}!")
```

Тук използваме unquote(name), за да включим стойността на аргумента name в израза. unquote позволява на стойността на name да бъде заменена в контекста на генерирания код.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

4. Изискване на модула

require MyMacros

Това казва на Elixir, че искаме да използваме макросите, определени в модула MyMacros. Без require, не можем да използваме макросите, дефинирани в този модул.

5. Извикване на макроса

MyMacros.say_hello("Elixir") # Изход: *Hello, Elixir!*

Този ред извиква макроса say_hello, предавайки му аргумент "Elixir". Когато извикваме макроса, кодът, който е дефиниран в quote, ще бъде генериран и изпълнен.

6. Стартиране на файла

Отваряне на терминала и навигиране до директорията, в която се намира файла. След това се изпълнява следната команда:

```
elixir my_macros.exs
```

Изход

Когато макросът say_hello("Elixir") бъде извикан, Elixir ще генерира код, който ще изпише "Hello, Elixir!" в конзолата

Чести грешки и проблеми

Често срещани проблеми:

- **Синтактични грешки:** Уверете се, че не е имало грешка при копирането или форматиране на кода.
- **Версия на Elixir:** Проверете дали версията на Elixir е актуална (може би 1.10 или по-нова).

Работа в iex:

Ако стартирате Elixir в iex, просто се записва кода в командната среда, но трябва да се уверите, че сте добавили `require MyMacros` за да можете да извиквате макроса.

```
iex>  
defmodule MyMacros do  
  defmacro say_hello(name) do  
    quote do  
      IO.puts("Hello, #{unquote(name)}!")  
    end  
  end  
end
```

```
{:module, MyMacros,  
<<70, 79, 82, ...>>,  
{:say_hello, 1}}
```

```
iex> require MyMacros  
iex> MyMacros.say_hello("Elixir")  
Hello, Elixir!
```

1.Модул: Започваме с дефиниция на модул с име MyMacros с ключовата дума defmodule.

2.Дефиниране на макрос: В defmacro създаваме макрос с име say_hello, който приема един аргумент name.

3.Quote: Ключовата дума quote е специален механизъм, който позволява на Elixir да вземе код и да го обработи като данни, вместо да го изпълни веднага. В случая, кодът, който искате да генерирате, е:

```
IO.puts("Hello, #{unquote(name)}!")
```

- **unquote:** С unquote(name) интегрираме стойността на аргумента name в стринга, който ще бъде изпълнен. Това е необходимо, за да може стойността на name да бъде заменена с конкретната стойност, която подавате при извикването на макроса.

Използване на макроса

MyMacros.say_hello("Elixir")

- **require:** С require MyMacros, вие казвате на Elixir, че искате да използвате макроса от модула MyMacros. Това е необходимо, за да можете да извикате макроса от текущия контекст.

Извикване на макроса: След това, извиквате макроса MyMacros.say_hello("Elixir"). При това извикване, Elixir ще генерира кода, който сте дефинирали в макроса, като замени #{unquote(name)} с "Elixir".

Изход: Резултатът от извикването на макроса е извеждането на текст на конзолата:
Hello, Elixir!

Резултатът, който се получава след дефиниция на модула MyMacros, е информация за модула и неговото съдържание в Elixir.

Структура на резултата

```
{:module, MyMacros, <<70, 79, 82, ...>>, {:say_hello, 1}}
```

1.:module: Това е атом, който указва, че информацията, която следва след него, е свързана с модул.

2.MyMacros: Това е името на модула, който току-що сме дефинирали. Неговото съдържание включва макроса say_hello, който можем да извикваме по-късно.

3.<<70, 79, 82, ...>>: Това е байтов масив, представляващ бинарната форма на модула. Всеки от числата в масива е ASCII код на символ, който съответства на определен шифър, направен от Elixir при компилацията на модула. В случая:

- 70, 79, 82 съответстват на буквите 'F', 'O', 'R', което е част от "FOR" в контекста на Elixir. Това обозначава, че модулът е компилиран и готов за използване.

4. {:say_hello, 1}: Тази част от резултата описва функцията (или макроса както е случая say_hello).

- :say_hello е името на макроса.
- 1 е ариостта на макроса, т.е. броят на аргументите, които той приема. В нашия случай, макросът say_hello приема един аргумент – name.

Извикване на макроса

След като сте дефинирали макроса и сте добавили require MyMacros, той е готов за използване:

```
iex> MyMacros.say_hello("Elixir")
```

quote превръща код в данни (AST)

```
quote do
```

```
  1 + 2
```

```
end
```

НЕ изчислява $1 + 2$, а връща структура (AST), например:

```
{:+, [context: Elixir, imports: [{1, Kernel}, {2, Kernel}]], [1, 2]}
```

quote = „взема кода и го запазва като структура“

unquote вмъква стойност в quote

Пример:

```
x = 5
```

```
quote do
```

```
  1 + unquote(x)
```

```
end
```

става: $1 + 5$

```
iex(1)> defmodule MyMacros do
  defmacro say_hello(name) do
    quote do
      IO.puts("Hello, #{unquote(name)}!")
    end
  end
end

{:module, MyMacros,
 <<70, 79, 82, 49, 0, 0, 7, 52, 66, 69, 65, 77, 65, 116, 85, 56, 0, 0, 0, 226,
 255, 255, 255, 235, 240, 69, 108, 105, 120, 105, 114, 46, 77, 121, 77, 97,
 99, 114, 111, 115, 128, 95, 95, 105, 110, 102, 111, 95, 95, 160, 97, 116,
 116, 114, 105, 98, 117, 116, 101, 115, 112, 99, 111, 109, 112, 105, 108, 101,
 160, 100, 101, 112, 114, 101, 99, 97, 116, 101, 100, 176, 101, 120, 112, 111,
 114, 116, 115, 95, 109, 100, 53, 144, 102, 117, 110, 99, 116, ...>>, ...}
```

unquote = „взема стойността и я поставя в кода“

```
iex(2)> quote do
  IO.puts("Hello, Elixir!")
end |> IO.inspect()
{:~, [], [{:__aliases__, [alias: false], [:IO]}, :puts]}, [],
["Hello, Elixir!"]}
{:~, [], [{:__aliases__, [alias: false], [:IO]}, :puts]}, [],
["Hello, Elixir!"]}
```

Макросите в Elixir ви позволяват да генерирате код по време на компилация, което е полезно за метапрограмиране и оптимизация.

Важно е да помните, че макросите работят по различен начин от функциите. Те не се изпълняват, а се преобразуват в код, който после се компилира. При неправилна употреба, макросите могат да направят кода по-труден за разбиране и поддръжка.

След като модулът `MyMacros` е зареден в `IEx`, вие можете многократно да извиквате неговите макроси или функции, без да е необходимо да ги дефинирате отново.

Ползи от макросите:

- **Разширяване на синтаксиса:** Макросите позволяват на разработчиците да създадат нови конструкции, които не са налични в основния синтаксис на Elixir.
- **Избягване на повторение:** С макросите можете да капсулирате сложна логика, която може да бъде повторно използвана на различни места.
- **Оптимизация на производителността:** Кодът, генериран от макроси, влияе на производителността на приложението, тъй като се компилира веднъж и не се изпълнява, както традиционните функции.

ОБОБЩЕНИЕ

Този код е пример за основна употреба на макроси в Elixir, демонстрирайки динамично генериране на код по време на компилация, за да улеснят задачите, свързани с повтаряеми шаблони или прилагането на метапрограмиране.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Създаване на потребителски функции

В Elixir можете да създавате свои функции, които разширяват функционалността на езика. Потребителските функции се дефинират с ключовата дума `def` в модули.

Определяне на функции:

Функциите могат да приемат аргументи и да връщат стойности. Всеки модул може да съдържа множество функции.

Пример:

```
defmodule Math do
  def add(a, b) do
    a + b
  end
end
```

```
IO.puts Math.add(5, 7) #result ще бъде 12
```

Нека да се допише модула, така, че да включва основните аритметични

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

```
defmodule Math do
  def add(a, b) do
    a + b
  end

  def subtract(a, b) do
    a - b
  end

  def multiply(a, b) do
    a * b
  end

  def divide(a, b) do
    if b == 0 do
      {:error, "Cannot divide by zero"}
    else
      a / b
    end
  end

  def run do
    IO.puts("Изберете операция:")
    IO.puts("1. Събиране")
    IO.puts("2. Изваждане")
    IO.puts("3. Умножение")
    IO.puts("4. Деление")
    IO.puts("0. Изход")
  end
end
```

```
choice = String.to_integer(IO.gets("Вашият избор: ") |> String.trim())
case choice do
  0 ->
    IO.puts("Изход от програмата.")
  n when n in 1..4 ->
    a = String.to_float(IO.gets("Въведете първото число: ") |>
String.trim())
    b = String.to_float(IO.gets("Въведете второто число: ") |>
String.trim())

    result =
      case choice do
        1 -> add(a, b)
        2 -> subtract(a, b)
        3 -> multiply(a, b)
        4 -> divide(a, b)
      end

    IO.puts("Резултат: #{result}")
    run() # повторно извикване на функцията, за да въвеждане на
ново действие

  _ ->
    IO.puts("Невалиден избор. Моля, опитайте отново.")
    run() # повторно извикване за нова опция
end
end
end
# Извикване на модул Math
Math.run()
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Функции с по-голяма гъвкавост:

Elixir позволява на функциите да имат различен брой аргументи и да бъдат избрани на базата на типовете на аргументите.

Пример:

```
defmodule Greeter do
  def greet(:morning) do
    "Good morning!"
  end
  def greet(:evening) do
    "Good evening!"
  end
end
```

```
greeting = Greeter.greet(:morning) # greeting ще бъде "Good morning!"
```

Обръщение към функция

За да се извикат функции в Elixir, трябва да се използва името на модула, следвано от името на функцията и аргументите, ако са налични. Elixir предоставя прост и интуитивен синтаксис за извикване на функции.

Пример:

```
defmodule Calculator do
  def multiply(a, b) do
    a * b
  end
end
```

```
result = Calculator.multiply(3, 4) # result ще бъде 12
```


Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Вградени функции

Elixir разполага с богат набор от вградени функции, които облекчават работата с основни типове данни и структури. Вградените функции в Elixir покриват множество области, включително манипулатори за списъци, низове, числа и т.н.

Списъци. Основни функции за работа със списъци

1. `hd/1`

Връща първия елемент на списъка.

Пример:

```
list = [1, 2, 3]
```

```
hd(list)
```

Връща 1

2. `tl/1`

Връща списъка без първия елемент.

Пример:

```
list = [1, 2, 3]
```

```
tl(list)
```

Връща [2, 3]

3. `length/1`

Връща броя на елементите в списъка.

Пример:

```
list = [1, 2, 3]
```

```
length(list)
```

Връща 3



4. `Enum.reverse/1`

Обръща елементите на списъка.

Пример:

```
list = [1, 2, 3]
```

```
Enum.reverse(list)
```

Връща [3, 2, 1]

5. `Enum.map/2`

Прилага функция към всеки елемент на списъка и връща нов списък.

Пример:

```
list = [1, 2, 3]
```

```
Enum.map(list, fn x -> x * 2 end)
```

Връща [2, 4, 6]

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

6. Enum.filter/2

Филтрира елементите на списъка по зададено условие.

Пример:

```
list = [1, 2, 3, 4]
Enum.filter(list, fn x -> rem(x, 2) == 0 end)
# Връща [2, 4]
```

7. Enum.reduce/3

Редуцира списъка до един резултат, прилагане на функция на двойки от елементи.

Пример:

```
list = [1, 2, 3]
Enum.reduce(list, 0, fn x, acc -> acc + x end)
# Връща 6
```

8. Enum.concat/1

Обединява списъци в един.

Пример:

```
list1 = [1, 2]
list2 = [3, 4]
Enum.concat([list1, list2])
# Връща [1, 2, 3, 4]
```

9. Enum.any?/2

Проверява дали поне един от елементите отговаря на условие.

Пример:

```
list = [1, 2, 3]
Enum.any?(list, fn x -> x > 2 end)
# Връща true
```



10. Enum.all?/2

Проверява дали всички елементи отговарят на условие.

Пример:

```
list = [1, 2, 3]
Enum.all?(list, fn x -> x > 0 end)
# Връща true
```

11. Enum.find/2

Намира първия елемент, който отговаря на условие.

Пример:

```
list = [1, 2, 3]
Enum.find(list, fn x -> x > 1 end)
# Връща 2
```

12. Enum.member?/2

Проверява дали елемент присъства в списъка.

Пример:

```
list = [1, 2, 3]
Enum.member?(list, 2)
# Връща true
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

13. List.insert_at/3

Вмъква елемент на определена позиция в списъка.

Пример:

```
list = [1, 2, 3]
```

```
List.insert_at(list, 1, 5)
```

```
# Връща [1, 5, 2, 3]
```

14. List.delete/2

Изтрива първото срещане на елемента от списъка.

Пример:

```
list = [1, 2, 3]
```

```
List.delete(list, 2)
```

```
# Връща [1, 3]
```

15. List.flatten/1

Плоско Flatten-ва (изравнява) вложените списъци.

Пример:

```
list = [1, [2, 3], 4]
```

```
List.flatten(list)
```

```
# Връща [1, 2, 3, 4]
```

16. List.first/1

Връща първия елемент на списъка (подобно на hd).

Пример:

```
list = [1, 2, 3]
```

```
List.first(list)
```

```
# Връща 1
```

17. List.last/1

Връща последния елемент на списъка.

Пример:

```
list = [1, 2, 3]
```

```
List.last(list)
```

```
# Връща 3
```



Низ. Основни функции за работа с низ

В Elixir, работа с низове (стрингове) е много популярна задача и предоставя разнообразие от функции за манипулиране. Низовете в Elixir са типично представени с UTF-8 кодирани символи и се използват главно с функции от модула String.

1. Дължина на низа

Функция: String.length/1

Предназначение: Връща броя на символите в низа.

Пример:

```
text = "Hello, world!"  
length = String.length(text)  
# length e 13
```

2. Извличане на подниз

Функция: String.slice/2

Предназначение: Връща част от низа, определена от начален индекс и дължина.

Пример:

```
text = "Hello, world!"  
substring = String.slice(text, 7, 5)  
# substring e "world"
```

3. Конкатенация (съединяване)

Функция: <> (оператор за съединяване)

Предназначение: Съединява два или повече низа.

Пример:

```
text1 = "Hello"  
text2 = "World"  
combined = text1 <> ", " <> text2 <> "!"  
# combined e "Hello, World!"
```



Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

4. Преобразуване в низ

Функция: Integer.to_string/1

Предназначение: Преобразува числа в низ.

Пример:

```
number = 123
```

```
text = Integer.to_string(number)
```

```
# text е "123"
```



5. Търсене на подниз

Функция: String.contains?/2

Предназначение: Проверява дали низът съдържа определен подниз.

Пример:

```
text = "Hello, world!"
```

```
contains_world = String.contains?(text, "world")
```

```
# true
```

6. Заменяне на подниз

Функция: String.replace/3

Предназначение: Заменя всички срещания на определен подниз с нов низ.

Пример:

```
text = "Hello, world!"
```

```
new_text = String.replace(text, "world", "Elixir")
```

```
# new_text е "Hello, Elixir!"
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

7. Преобразуване на низ

Функции: String.upcase/1, String.downcase/1, String.capitalize/1

Предназначение: Преобразува низа в главни (upcase), малки (downcase) или главни букви в началото (capitalize) букви.

Пример:

```
text = "hello, world. hell pepi!"
```

```
IO.puts(String.upcase(text))
```

```
IO.puts(String.downcase(text))
```

```
IO.puts(String.capitalize(text))
```

Output:

```
HELLO, WORLD. ***** PEPI!hello, world. ***** pepi!Hello, world. ***** pepi!
```



8. Разделяне на низ

Функция: String.split/2

Предназначение: Разделя низа на поднизове на базата на определен разделител.

Пример:

```
text = "Hello, world!"
```

```
words = String.split(text, ", ")
```

```
# words e ["Hello", "world!"]
```

9. Обединяване на низове

Функция: Enum.join/2

Предназначение: Обединява елементите на списък в един низ, използвайки определен разделител.

Пример:

```
words = ["Hello", "world"]
```

```
sentence = Enum.join(words, " ")
```

```
# sentence e "Hello world"
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

10. Проверка на наличност

Функции: `String.starts_with?/2`, `String.ends_with?/2`

Предназначение: Проверяват дали низът започва или завършва с определен подниз.

Пример:

```
text = "Hello, world!"  
starts_with_hello = String.starts_with?(text, "Hello") # true  
ends_with_world = String.ends_with?(text, "world!") # true
```

Тези функции и методи за работа с низове в Elixir предоставят основните инструменти за манипулиране на текстови данни. Разбирането на тези функции е важно за разработчици, които работят с текстова обработка, анализ на данни и много други приложения в Elixir.

11. Извличане на символ по индекс

Функция: `String.at/2`

Предназначение: Връща символа на указан индекс в низа.

Пример:

```
text = "Hello"  
char_at = String.at(text, 1) # char_at е "e"
```

12. Проверка за празен низ

Функция: `String.empty?/1`

Предназначение: Проверява дали низът е празен (има нулева дължина).

Пример:

```
text = ""  
is_empty = String.empty?(text)  
# is_empty е true
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

13. Премахване на „спейс“

Функция: `String.trim/1`

Предназначение: Премахва водещи и завършващи „бели знаци“ от низа.

Пример:

```
text = " Hello, world! "
```

```
trimmed = String.trim(text) # trimmed е "Hello, world!"
```

14. Проверка за наличност само на букви

Функция: `String.match?/2`

Предназначение: Проверява дали низа съвпада с определен регулярен израз.

Пример:

```
text = "Hello"
```

```
is_alpha = String.match?(text, ~r/^\p{L}+$/) # is_alpha е true (само букви)
```


Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

```
text = "Hello"
```

```
is_alpha = String.match?(text, ~r/^\p{L}+$/ ) # is_alpha е true (само букви)
```

```
text = "Hello":
```

Тук създаваме променлива text, на която присвояваме стойността "Hello". Тази стойност представлява низ (стринг), който съдържа буквите H, e, l, l, o.

String.match?:

Това е функция от модула String, която проверява дали низ съвпада с даден регулярен израз. Функцията връща true или false.

Изразът String.match?(text, ~r/^\p{L}+\$/) проверява дали текстът отговаря на определен шаблон (регулярен израз).

~r/^\p{L}+\$/:

Това е синтаксисът за регулярен израз в Elixir. Той започва с ~r (нарича се още сигил) и следва шаблонът.

Регулярния израз:

~r/.../ - означава, че имаме регулярен израз, а чертите указват границите му, т.е. наклонената черта е разделител, не е част от регулярния израз

^: Тази част указва начало на низа. Тоест, проверката започва от самото начало на текста.

\p{L}: Този патерн означава "буква". В \p{L} \p е познато като "клас символи" (property class), а {L} означава "букви".

Следователно \p{L} съвпада с всяка буква от азбуката, независимо от езика (латински, кирилица и др.).

+: Този символ указва, че предходният елемент (\p{L}) трябва да се появи **поне веднъж**. Тоест, търсим една или повече букви последователно.

\$: Тази част указва край на низа. Тоест, проверката завършва в края на текста.

Какво прави is_alpha

Когато извикате String.match?(text, ~r/^\p{L}+\$/), Elixir проверява дали целият низ text ("Hello") се състои само от букви. В нашия случай:

Низът "Hello" е съставен изцяло от букви.

Патернът ~r/^\p{L}+\$/ проверява дали низът започва и завършва с букви, без интервали или други символи.

Тъй като "Hello" отговаря на поставените условия, is_alpha ще бъде true.

Резултат:

Ако след изпълнението на кода проверите стойността на is_alpha, тя ще бъде: true.

Така, кодът проверява и потвърждава, че текстът съдържа само букви, и присвоява резултата на променливата is_alpha.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

15. Преобразуване в списък от символи

Функция: `String.graphemes/1`

Предназначение: Преобразува низа в списък от графеми (символи).

Пример:

```
text = "Hello"
```

```
graphemes = String.graphemes(text)
```

```
# graphemes е ["H", "e", "l", "l", "o"]
```

16. По-сложна замяна на подниз

Функция: `String.replace/3` или 4 arguments

Предназначение: Заменя поднизове, базирани на функции или условие.

Синтаксис:

```
String.replace(string, pattern, replacement)
```

Пример:

```
text = "Hello, world!"
```

```
new_text = String.replace(text, "world", fn _ -> "Elixir" end)
```

```
# new_text е "Hello, Elixir!"
```

Синтаксис:

`String.replace(string, pattern, replacement, options)`, където options е **keyword list**, като `global: true/false` и `case: :sensitive / :insensitive`

Пример:

```
String.replace(string, pattern, replacement, options \\ [])
```

```
text = "one one one"
```

```
String.replace(text, "one", "two", global: false)
```

резултат: **"two one one"**, защото `global: false`, ще замени само първото срещане (match)



17. Проверка на валидност на UTF-8

Функция: `String.valid?/1`

Предназначение: Проверява дали низът е валиден UTF-8.

Пример:

```
binary = <<255, 254, 0>>
```

```
is_valid = String.valid?(binary) # is_valid е false
```

binary = <<255, 254, 0>>:

Тук създаваме бинарна стойност, която е представена с използването на синтаксиса `<< >>`, който се използва за работа с бинарни данни в Elixir.

В конкретния случай, `<<255, 254, 0>>` представлява бинарна стойност с три байта.

255 - байт с 8 бита, изцяло запълнен (11111111 в двоична форма).

254 - байт с 8 бита, който е един по-малко от 255 (11111110 в двоична форма).

0 - байт, който е изцяло нулев (00000000 в двоична форма).

String.valid?(binary):

Тази функция проверява дали подадената стойност е валиден UTF-8 низ. Всъщност, функцията `String.valid?/1` проверява, дали бинарната стойност може да бъде интерпретирана като текст на формата UTF-8.

В Elixir, низовете (strings) са представени в UTF-8. Това означава, че всеки символ в низ е кодиран в байтове, които следват стандартите на UTF-8.

Какво прави `is_valid`

Когато извикате `String.valid?(binary)`, Elixir проверява дали бинарната стойност, предоставена на функцията, може да бъде правилно интерпретирана като валиден UTF-8 низ.

В случая:

Бинарната стойност `<<255, 254, 0>>` съдържа байтове, които не съответстват на валидни UTF-8 символи.

Специфичното нещо е, че стойността 255 (и 254) не е валиден код на UTF-8 символ. UTF-8 може да представи само определен диапазон от байтове. Броевете, по-големи от 127, често се използват в контекста на много байтови енкодвания (например, символи от диаграмата на Unicode), но 255 конкретно не е символ в UTF-8.

Тъй като бинарното представяне не съдържа валидни UTF-8 символи, функцията `String.valid?/1` ще върне `false`.

Резултатът от изпълнението на кода, ако проверите стойността на `is_valid`, тя ще бъде: `false`

18. Подмяна на подниз в зависимост от условие

Функция: String.replace/3 с :global

Предназначение: Заменя всички срещания на подниз с условен механизъм.

Пример:

```
text = "Hello, world! World is great!"
```

```
new_text = String.replace(text, "World", "Elixir", global: true) # new_text е "Hello, world! Elixir is great!"
```

Контролът чрез options, включва:

- global: true (за всички стойности)
- global: false (само първата стойност)
- global: true е излишно, тъй като поведението е по подразбиране

Или като обобщение

Синтаксис

/3

/4 + global: true

/4 + global: false

Поведение

винаги заменя всички

същото като /3

заменя само първия

19. Намиране на индекс на подниз

Функция: String.index/2

Предназначение: Връща индекса на първото срещане на подниз в низа или nil, ако не е намерен.

Пример:

```
text = "Hello, world!"
```

```
index = String.index(text, "world") # index е 7
```

20. Разделяне на низа с регулярни изрази

Функция: String.split/2 с регулярни изрази

Предназначение: Разделя низа на базата на регулярни изрази.

Пример:

```
text = "one, two; three four"
```

```
parts = String.split(text, ~r/[,;\s;]+/) # parts е ["one", "two", "three", "four"]
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

```
text = "one; two; three four"
parts = String.split(text, ~r/[,\\s;]+/) # parts е ["one", "two", "three", "four"]
```



Този регулярен израз е предназначен за разделяне на низ на базата на запетая, бели знаци или точка и запетая.

Какво означава всеки от символите включени в регулярния израз?

~r - Това е нотация в Elixir, която указва, че следващият текст е регулярен израз. То помага на Elixir да разбере, че ще се работи с регулярни изрази.

[и] - Тези символи обозначават клас от знаци. Всички знаци, вътре в квадратните скоби, се разглеждат като валидни елементи на съвпадението. Във вътрешността на [и] можем да укажем множество алтернативи.

, - Самата запетая е знак, който указва, че низа ще бъде разделен и при среща на този символ. Той трябва да се появи в низа, за да бъде извършено разделяне.

\\s - Този символ е специален и обозначава произволен бял знак (пробел, таб, нов ред и т.н.). Той позволява на регулярния израз да съвпадне с всички видове бели знаци.

; - Подобно на запетаята, точката и запетаята също действа като разделител в низа. Всички такива символи, вмъкнати в клас от знаци, указват, че когато срещнем точка и запетая, низът трябва да бъде разделен.

+ - Този символ е квантификатор, който указва, че предшестващият елемент (когато е изписан в квадратни скоби) трябва да се срещне поне веднъж, но може да се среща многократно. В този случай, [, \\s;] може да се среща един или повече пъти за извършване на разделяне.

Как работи целият израз?

Изразът ~r/[,\\s;]+/ указва, че всяко срещане на запетая, бял знак или точка и запетая, независимо от това колко често се появяват, ще причини разделяне на низа.

Например, за входен низ "one, two; three four", регулярният израз ще раздели текста на: ["one", "two", "three", "four"].

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Числови функции

В Elixir съществуват множество функции за работа с числа.

1. abs/1

Предназначение: Връща абсолютната стойност на числото.

Пример:

```
abs_value = abs(-10)  
# abs_value е 10
```

2. round/1

Предназначение: Закръгля числото до най-близкото цяло число.

Пример:

```
rounded_value = round(3.6)  
# rounded_value е 4
```

3. floor/1

Предназначение: Връща най-близкото цяло число, по-малко или равно на текущата стойност.

Пример:

```
floor_value = floor(3.6)  
# floor_value е 3
```

4. ceil/1

Предназначение: Връща най-близкото цяло число, по-голямо или равно на текущата стойност.

Пример:

```
ceil_value = ceil(3.2)  
# ceil_value е 4
```

5. min/2

Предназначение: Връща по-малкото от две числа.

Пример:

minimum = min(5, 10)

minimum е 5

6. max/2

Предназначение: Връща по-голямото от две числа.

Пример:

maximum = max(5, 10)

maximum е 10

7. div/2

Предназначение: Извършва целочислено деление на две числа.

Пример:

division = div(10, 3)

division е 3

8. rem/2

Предназначение: Връща остатъка от делението на две числа.

Пример:

remainder = rem(10, 3)

remainder е 1

9. pow/2

Предназначение: Степенува първото число с второто.

Пример:

power = :math.pow(2, 3)

power е 8.0 (върнатото е с плаваща запетая)

10. sqrt/1

Предназначение: Връща квадратния корен на дадено число.

Пример:

```
square_root = :math.sqrt(16)  
# square_root е 4.0
```

Сума и произведение на числа:

```
sum = 1 + 2 + 3 + 4 # sum е 10  
product = 3 * 4 # product е 12
```

11. sin/1

Предназначение: Връща синуса на зададения ъгъл в радиани.

Пример:

```
sine_value = :math.sin(:math.pi() / 2)  
# sine_value е 1.0
```

12. cos/1

Предназначение: Връща косинуса на зададения ъгъл в радиани.

Пример:

```
cosine_value = :math.cos(:math.pi())  
# cosine_value е -1.0
```

13. tan/1

Предназначение: Връща тангенса на зададения ъгъл в радиани.

Пример:

```
tangent_value = :math.tan(:math.pi() / 4)  
# tangent_value е 1.0
```


В Elixir, няма вградена директна функция за котангенс (cotangent) в модула :math, но можете да го изчислите, използвайки тангенс. Котангенсът на ъгъл е обратното на тангенс, т.е. $\cot[f_0](\theta) = 1 / \tan[f_0](\theta)$

Изчисляване на котангенс в Elixir

Можете да дефинирате собствена функция за котангенс, която използва :math.tan/1 за изчисление.

```
defmodule Trigonometry do
  def cotangent(angle_in_radians) do
    1 / :math.tan(angle_in_radians)
  end
end
```

Пример за употреба

```
angle_in_radians = :math.pi() / 4 # 45 градуса
cotangent_value = Trigonometry.cotangent(angle_in_radians)
# cotangent_value ще бъде 1.0
IO.puts("Котангенсът от 45 градуса е: #{cotangent_value}")
```

Обяснение на примера

В дефинирания модул Trigonometry, функцията cotangent/1 приема един аргумент (ъгъл в радиани).

Функцията връща обратния тангенс на зададения ъгъл.

Примерът показва как да се изчисли котангенсът на 45 градуса ($\pi/4$ радиана), който е равен на 1.0.

По този начин можете да изчислите котангенс в Elixir, като просто комбинирате наличната функция за тангенс.

14. log/1

Предназначение: Връща натуралния логаритъм на число.

Пример:

```
log_value = :math.log(2.71828) # log_value прибл. е 1.0
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

15. log10/1

Предназначение: Връща логаритъма с основа 10 на число.

Пример:

```
log10_value = :math.log10(100) #log10_value е 2.0
```

16. exp/1

Предназначение: Връща стойността на е на степен x.

Пример:

```
exp_value = :math.exp(1) #exp_value прибл. е 2.71828
```

17. factorial/1

Предназначение: Връща факториела на дадено число (не е в стандартната библиотека, но можем да го реализираме).

Пример на функция:

```
def factorial(0), do: 1
def factorial(n) when n > 0, do: n * factorial(n - 1)
fact_value = factorial(5) #fact_value е 120
```

18. sum/1

Предназначение: Сумата на елементите в списък.

Пример:

```
sum_value = Enum.sum([1, 2, 3, 4]) #sum_value е 10
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

22. is_number/1

Предназначение: Проверява дали входната стойност е число (цяло или с плаваща запетая).

Пример:

```
result = is_number(5) # result е true
```

23. to_string/1

Предназначение: Преобразува числото в низ.

Пример:

```
str_value = to_string(42) # str_value е "42"
```

24. String.to_integer/1

Предназначение: Преобразува низ в цяло число.

Пример:

```
int_value = String.to_integer("123") # int_value е 123
```

25. String.to_float/1

Предназначение: Преобразува низ в число с плаваща запетая.

Пример:

```
float_value = String.to_float("3.14") # float_value е 3.14
```

26. Integer.gcd/2

Предназначение: Връща най-голямото общо кратно на две цели числа.

Пример:

```
gcd_value = Integer.gcd(48, 18) # gcd_value е 6
```

27. Integer.pow/2

Предназначение: Повдига цяло число на степен цяло число.

Пример:

```
power_value = Integer.pow(2, 3)
```

power_value е 8

28. :math.hypot/2

Предназначение: Връща хипотенузата на правоъгълен триъгълник, зададен с две страни.

Пример:

```
hypot_value = :math.hypot(3, 4)
```

hypot_value е 5.0

29. :math.atan2/2

Предназначение: Връща ъгъла в радиани от координатите (y, x).

Пример:

```
angle = :math.atan2(3, 4)
```

angle е приблизително 0.6435

30. :math.deg/1

Предназначение: Преобразува радиани в градуси (изисква добавяне на модула :math).

Пример:

```
degrees = :math.deg(:math.pi() / 2) # degrees е 90
```

Тези функции предоставят широка гама от операции и проверки, свързани с числата, които могат да бъдат полезни в различни сценарии на програмиране в Elixir.

ОБОБЩЕНИЕ

Оператори, макроси, потребителски функции и вградени функции оформят основата на всеки проект, написан на Elixir. Разработчиците могат да използват мощни инструменти за работа с данни и създаване на приложения, които са както ефективни, така и лесни за поддържане. Със своята функционалност, гъвкавост и мощни макроси, Elixir остава един от най-предпочитаните езици в света на разработката на приложения.



Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Често използвани потребителски функции

```
defmodule Utilities do
```

Функция за сумиране на две числа

```
def sum(a, b) do
```

```
  a + b
```

```
end
```

Функция за проверка на четно или нечетно число

```
def even_or_odd(number) when is_integer(number) do
```

```
  if rem(number, 2) == 0 do
```

```
    "#{number} е четно."
```

```
  else
```

```
    "#{number} е нечетно."
```

```
  end
```

```
end
```

Функция за изчисляване на факториел

```
def factorial(0), do: 1
```

```
def factorial(n) when n > 0 do
```

```
  n * factorial(n - 1)
```

```
end
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Функция за конвертиране на температура от Целзий в Фаренхайт

```
def celsius_to_fahrenheit(celsius) do
  (celsius * 9 / 5) + 32
end
```

Функция за намиране на максималното число в списък

```
def max_in_list([], do: nil)
def max_in_list(list) do
  Enum.max(list)
end
```

Функцията `max_in_list/1` връща най-голямата стойност в списък. Ако списъкът е празен, връща `nil`.

Функция за стилизиране на стринг (например, превръщане в главни букви)

```
def stylize_string(string) do
  String.upcase(string)
end
```

Функция за обърнато изписване на стринг

```
def reverse_string(string) do
  String.reverse(string)
end
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Функция за проверка дали дума е палиндром

```
def is_palindrome(word) do
  normalized_word = String.downcase(word)
  normalized_word == String.reverse(normalized_word)
end
```

Функцията проверява дали дадена дума е палиндром, като първо я преобразува в малки букви. След това сравнява думата с нейния обърнат вариант и връща true или false.

Функция за изчистено показване на грешка

```
def show_error(message) do
  IO.puts("Грешка: #{message}")
end
```

Функция за генериране на произволно число в зададен диапазон

```
def random_number(min, max) do
  :rand.uniform(max - min + 1) + min - 1
end
```

end

Тази функция `random_number/2` генерира случайно цяло число между `min` и `max`, включително и двете граници. Тя използва `:rand.uniform/1`, като коригира стойността, за да започва от `min` вместо от 1.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Използваните функции имат следното предназначение:

sum(a, b): Сумира две числа.

even_or_odd(number): Проверява дали дадено число е четно или нечетно.

factorial(n): Изчислява факториела на число n (с рекурсивен подход).

celsius_to_fahrenheit(celsius): Конвертира температура от Целзий в Фаренхайт.

max_in_list(list): Намира максималното число в списък.

stylize_string(string): Стилизира стринг, например, трансформира го в главни букви.

reverse_string(string): Обърнато изписва стринг.

is_palindrome(word): Проверява дали дума е палиндром, игнорирайки главни букви.

show_error(message): Показва диагностично съобщение за грешка.

random_number(min, max): Генерира произволно число в зададения диапазон. Функцията random_number/2 изисква параметри min и max, за да генерира произволно число в посочения диапазон.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

2.1. Разбиране на времето на изпълнение

Времето на изпълнение в Elixir се отнася до моментите, когато кодът на Elixir се стартира, интерпретира и изпълнява. Elixir работи върху Erlang VM (BEAM), който предлага управляемост и висока производителност. Кодът на Elixir е компилиран в байт-код, който изпълнява Erlang VM в реално време.

2.2. Модули и функции по време на изпълнение

Модулите в Elixir са основните структури за организация на кода. Функции се дефинират в модули и могат да бъдат извиквани по време на изпълнение. Elixir поддържа динамично свързване на модули и функции, което позволява на програмите да се адаптират в реално време.

Всеки модул е пространство от имена, което групира функции и структурни данни. Функциите в модулите могат да бъдат извиквани по всяко време по време на изпълнение, след като модулът е зареден.

```
defmodule Math do
```

```
  def add(a, b) do
```

```
    a + b
```

```
  end
```

```
  def subtract(a, b) do
```

```
    a - b
```

```
  end
```

```
end
```

```
# Извикване на функции
```

```
sum = Math.add(3, 4)      # sum = 7
```

```
difference = Math.subtract(10, 4) # difference = 6
```

При стартиране на програмата, Elixir компилира модулите и ги зарежда в паметта, което означава, че можете да извиквате функции, както е показано. Функциите могат да имат различни реализации чрез шаблони и позволяват дефиниране на различни входни стойности.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

2.3. Чисти модули Erlang

Чистите модули на Erlang представляват модули, написани на Erlang, които могат да бъдат използвани в Elixir. Те предлагат API, който е надежден и предсказуем при управлението на паралелизъм.

Чистите модули не съдържат специфични функции на Elixir и следва стандартите на Erlang. Тъй като Elixir работи върху Erlang VM (BEAM), той е съвместим с Erlang и може да използва тяхната функционалност.

Пример: да предположим, че имаме модул на Erlang, който предлага основни функции за работа с числа. Можете да го използвате директно в Elixir, без никакви модификации:

```
-module(math_utils).
```

```
-export([add/2, subtract/2]).
```

```
add(A, B) -> A + B.
```

```
subtract(A, B) -> A - B.
```

След това можете да извикате тези функции в Elixir:

```
result = :math_utils.add(5, 7) # result = 12
```

2.4. Динамично извикване на функции

В Elixir е възможно да се извикват функции динамично чрез `apply/3`, където можете да посочите името на модула, името на функцията и списък с аргументи.

```
apply(ModuleName, :function_name, [arg1, arg2])
```

Пример:

```
module_name = Math
```

```
function_name = :add
```

```
arguments = [5, 10]
```

```
result = apply(module_name, function_name, arguments) # result = 15
```

Тази функция е полезна, когато искате да извикате функция, но името ѝ не е известно по време на компилация.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

2.5. Стартиране на времето за изпълнение

Elixir стартира изпълнението на програмата, като инициализира Erlang VM. Командата `elixirc` компилира Elixir файлове и `iex` стартира интерактивната обвивка за разработка.

Можете да стартирате Elixir с изпълнението на командата:

```
elixir my_script.exs
```

или за интерактивно среда:

```
iex
```

Това стартира REPL среда, където можете да пишете Elixir код и да го изпълнявате в реално време.

2.6. Интерактивна обвивка

`iex` (Interactive Elixir) е интерактивна обвивка, която позволява на разработчиците да изпълняват Elixir код на място, да тестват функции и да работят в реално време.

Основни характеристики на IEx:

Изпълняване на Elixir код: Можете да изпълнявате Elixir функции и модулни интерактивно.

Зареждане на модули: Можете да заредите модули с `c("file.ex")` команда.

Поддръжка за дебъгване: IEx предлага опции за настройка на дебъгер и ползване на `h (help)` за справка.

2.7. Изпълнение на скриптове

Скриптове в Elixir могат да бъдат изпълнявани директно от командния ред, например с `elixir script.exs`. Те се интерпретират и не изискват предварителна компилация.

Пример за прост скрипт:

```
# my_script.exs
```

```
IO.puts("Hello, World!")
```

Изпълнявате скрипта, като стартирате командата: `elixir my_script.exs`

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

2.8. Инструментът за смесване

Mix е инструмент за управление на проекти в Elixir, който улеснява компилирането, тестването, зависимостите и много други аспекти на разработката на Elixir приложения.

Mix е инструмент за управление на проектите в Elixir. Той предлага набор от инструменти, които подпомагат разработчиците в компилирането на проекти, управление на зависимости, стартиране на тестове и изграждане на документация.

Основни команди на Mix:

- **Създаване на нов проект:** `mix new project_name`
- **Стартиране на приложението:** `mix run`
- **Управление на зависимости:** Добавяте зависимости в `mix.exs` файл и след това изпълнявате `mix deps.get`.
- Mix е изключително полезен по време на разработка, тъй като обединява множество функции и опции в един инструмент.

2.9. Макроси

Макросите в Elixir са механизъм, който позволява на разработчиците да генерират код по време на компилация, подобен на шаблоните в други езици. Те ви позволяват да пишете по-абстрактен и по-малко повтарящ се код, като автоматично генерират нужния Elixir код, основан на входните параметри. Макросите работят на ниво синтаксис и могат да манипулират AST (Abstract Syntax Tree) на Elixir. Макросите се дефинират с ключовата дума `defmacro`, а не с `def`, която се използва за обикновени функции.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

2.9.1. Създаване на първият макрос

Да видим как да създадем основен макрос. Например, можем да направим макрос, който генерира функция за поздрав:

```
defmodule Greetings do
  defmacro greet(name) do
    quote do
      IO.puts("Hello, #{unquote(name)}!")
    end
  end
end
```

Използване на макроса

```
defmodule Test do
  require Greetings
  Greetings.greet("Alice")
End
```

В примера дефинираме модул `Greetings`, в който създаваме макрос `greet`. Макросът `greet` приема `name` и генерира код, който извежда поздрав на конзолата. Използваме `require Greetings` в модул `Test`, за да можем да извикаме макроса. Когато извикате `Greetings.greet("Alice")`, той ще генерира код и ще извика `IO.puts("Hello, Alice!")`.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

2.9.2. Макро хигиена

Макрото хигиена в Elixir означава, че макросите автоматично управляват обхвата на имената, за да избегнат конфликт между идентификаторите. Когато използвате `unquote`, можете да включите локални променливи в генерирания код, а не глобални или чужди идентификатори. Основната идея е, че макросите трябва да бъдат изолирани, за да не повлияят на локалните променливи и функции в контекста, във който са извикани. В Elixir, хигиената на макросите е автоматична.

Пример:

```
defmodule Example do
  defmacro example_macro do
    variable = "I am a local variable"
    quote do
      IO.puts(unquote(variable))
    end
  end
end

defmodule Test do
  require Example
  Example.example_macro() # Извежда: I am a local variable
end
```

В примера `variable` се обработва с `unquote`, което означава, че локалният контекст остава изолиран.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

2.9.3. Околната среда

В контекста на макросите, "околната среда" се отнася до информацията, която е достъпна за макроса, когато се компилира. Чрез макроси можете да получите достъп до важна информация за контекста, в който те са извикани, включително детайли за модули, функции и променливи.

Можете да получите информация за контекста, в който се изпълняват, чрез `__MODULE__`, `__CALLER__` и други макроси, които предоставят допълнителна информация.

Пример:

```
defmodule Environment do
  defmacro print_caller_info do
    quote do
      caller = __CALLER__
      IO.inspect(caller) # Извежда информация за модула и позицията
    end
  end
end
```

```
defmodule Test do
  require Environment
  Environment.print_caller_info()
end
```

Когато извикате `Environment.print_caller_info()`, ще получите информация за модула и реда, на който е извикан макросът.

2.9.4. Частни макроси

Частните макроси се дефинират с `defmacro` и не са достъпни извън модула. Това е полезно, когато искате да разделите интерфейса на вашия модул от вътрешната логика. Частните макроси могат да се използват само в модулите, в които са дефинирани.

Пример:

```
defmodule MyMacros do
  defmacro private_macro do
    quote do
      IO.puts("This is a private macro.")
    end
  end
end
```

```
def public_macro do
  private_macro()
end
end
```

```
defmodule Test do
  require MyMacros
```

```
  # MyMacros.private_macro() # Това ще доведе до грешка, тъй като е частен
  MyMacros.public_macro() # Това работи
end
```

В примера `private_macro` не може да бъде извикан извън модул `MyMacros`, за разлика от `public_macro`, който може да бъде извикан и използва частния макрос. Частните макроси предлагат механизъм за инкапсулиране на логиката и осигуряване на контролиран достъп до определени функции в модул.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Заклучение

Макросите в Elixir предоставят мощен инструмент за кодова генерация и абстракция. Те предлагат различни концепции, включително хигиена, работа с контекста и частност, които са важни за собствената организация и структура на кодовете. Когато се използват правилно, те могат да направят кода по-гладък и по-лесен за разбиране.

3. Създаване на потребителски функции

Elixir позволява създаването на функции с подходяща синтаксична структура.

Потребителските функции в Elixir представляват основополагаеща част от програмирането на Elixir. Те ви позволяват да организирате логиката на вашето приложение и да избягвате дублиране на код.

3.1. Обръщение към функция

Обръщението към функция в Elixir става чрез името на функцията, последвано от аргументите в скоби. Функциите в Elixir са дефинирани в модули.

```
MyModule.my_function(arg1, arg2)
```

Пример за сума на 2 числа:

```
defmodule Math do
```

```
  def add(a, b) do
```

```
    a + b
```

```
  end
```

```
end
```

```
# Извикване на функция
```

```
result = Math.add(5, 7) # result = 12
```

Когато извикате `Math.add(5, 7)`,

Elixir ще намери функцията `add` в модула `Math` и ще извърши операцията.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.2. Вградени функции

Elixir идва с множество вградени функции, например Enum.map/2, List.first/1.

Elixir предлага много вградени функции, които можете да използвате за различни цели. Например, можете да използвате Enum, List, Map и др. Те предлагат функции за работа с колекции, стрингове и различни типове данни.

Примери за вградени функции:

Използване на Enum

```
list = [1, 2, 3]
```

```
doubled = Enum.map(list, fn x -> x * 2 end) # doubled = [2, 4, 6]
```

Използване на String

```
string = "hello"
```

```
capitalized = String.upcase(string) # capitalized = "HELLO"
```

Вградените функции са допълнение на вашия код и могат да спестят много време.

3.3. Function type annotations

Elixir не поддържа явни анотации за типове, но има поддръжка за "type specs", написани с @spec. Това е по-скоро за помощ на разработчици, отколкото за проверка на типове по време на компилация.

Пример:

```
@spec add(integer, integer) :: integer
```

```
def add(a, b) do
```

```
  a + b
```

```
end
```

Пример:

```
defmodule Calculator do
```

```
  @spec add(number, number) :: number
```

```
  def add(a, b) do
```

```
    a + b
```

```
  end
```

```
end
```

В примера @spec предоставя информация за типовете на аргументите и на връщаната стойност на функцията.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.4. Function heads

Функциите могат да имат различни "глави" в зависимост от подадените аргументи. Функционалните глави се отнасят до различните реализации на една функция чрез шаблони. В Elixir можете да имате функции с идентично име, но с различен брой или тип аргументи.

Пример:

```
def greet(:morning), do: "Good morning"def greet(:evening), do: "Good evening"
```

Пример:

```
defmodule Math do
  def add(a, b) do
    a + b
  end

  def add(a) when is_integer(a) do
    a + 1 # добавя 1, ако аргументът е едно число
  end
end
```

```
Math.add(5, 7) # Извежда: 12
```

```
Math.add(5)    # Извежда: 6
```

В горния пример функцията add има две глави - една приема две числа, а другата приема едно число.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.5. *Function overloading*

Elixir не поддържа класическо претоварване на функции, но можете да използвате шаблони за аргументите, за да постигнете подобно поведение.

т.е. Elixir не можете да имате функции с абсолютно същото име и идентични параметри, но можете да постигнете подобно поведение чрез различни функции, базирани на шаблони.

Пример:

```
defmodule Example do
  def greet(name) do
    "Hello, #{name}!"
  end
```

```
  def greet(name, age) do
    "Hello, #{name}! You are #{age} years old."
  end
end
```

Извикване на функции

```
Example.greet("Alice")    # Извежда: "Hello, Alice!"
```

```
Example.greet("Bob", 30)  # Извежда: "Hello, Bob! You are 30 years old."
```

Този механизъм не е точно "претоварване" по начина, по който бихте го намерили в езици на базата на C, но същността му е подобна.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.6. Referencing functions

Можете да се позовавате на функции в модули с помощта на `&`.

```
my_function = &MyModule.my_function/1
```

Можете да съхранявате функции в променливи и да се обръщате към тях по-късно.

Elixir използва подхода с "първокласни функции", което означава, че функциите могат да бъдат присвоявани в променливи, предавани като аргументи и т.н.

Пример: `add = fn a, b -> a + b end`
`result = add.(3, 5) # Извежда: 8`

В горния пример `add` е анонимна функция, която е присвоена на променлива. Можете да се обърнете към анонимната функция, използвайки оператора `&`.

3.7. Calling anonymous functions

Анонимни функции (lambda функции) се извикват по следния начин:

```
add = fn a, b -> a + b end  
result = add.(1, 2) # result e 3
```

Анонимните функции в Elixir се дефинират с `fn` и завършват с `end`. Те могат да бъдат извиквани чрез оператора `&`

Пример:

```
square = fn x -> x * x end  
result = square.(4) # Извежда: 16
```

Ако искате да предадете анонимна функция на друга функция, можете да го направите по следния начин:

```
list = [1, 2, 3]  
squared_list = Enum.map(list, fn x -> x * x end)  
# [1, 4, 9]
```

3.8. Labelled arguments

Elixir не поддържа обозначени аргументи, както някои други езици (например Python), но можете да постигнете подобно поведение чрез структури или с помощта на `map`.

Пример с `map`:

```
defmodule User do
  def create_user(%{name: name, age: age}) do
    "User #{name} is #{age} years old."
  end
end
```

```
user_info = %{name: "Alice", age: 30}
```

```
User.create_user(user_info) # Извежда: "User Alice is 30 years old."
```

Мапа предлага гъвкава структура за работа с множество "обозначени" аргументи.

Заклучение

Създаването на потребителски функции в Elixir предоставя гъвкавост и възможност за организиране на код. Параметрите като анотации, функционалните глави и справочните функции предлагат мощни инструменти, които помагат за създаването на модулен и четим код.

3.9. Анонимни функции

Анонимните функции в Elixir (или "функции без име") се дефинират с ключовите думи `fn` и `end`. Те не са прикрепени към модул или име и могат да се предават директно на други функции или да се присвояват на променливи.

Пример за анонимна функция:

```
add = fn a, b -> a + b end
```

```
result = add.(3, 5) # Извежда: 8
```

Анонимните функции могат да приемат и опционални аргументи и се използват често в контекста на обработка на списъци или колекции, например с `Enum.map/2`.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.10. Capture Operator

Операторът на улавяне (&) предоставя начин за опростяване на създаването на анонимни функции, като позволява директно извикване на именувани функции. Можете да използвате оператора, за да запишете функция, базирана на друга функция, и да избегнете необходимостта от fn блок.

Пример:

```
add = &(&1 + &2) # &1 и &2 представляват аргументите
result = add.(3, 5) # Извежда: 8
```

Използване на оператор на улавяне за съществуваща функция

```
square = &(&1 * &1)
result_square = square.(4) # Извежда: 16
```

3.11. Multiple Implementations

Elixir поддържа различни реализации на функции чрез шаблониране, всяка функция може да има множество глави. т.е. Elixir позволява имплементирането на функции с множество глави (функционални глави) както с именувани функции, така и с анонимни функции. В случая на именуваните функции, можете да имате различни реализации на една и съща функция в зависимост от аргументите.

Пример:

```
defmodule Math do
  def calculate(1), do: "One"
  def calculate(2), do: "Two"
  def calculate(_), do: "Other"
end
```

```
Math.calculate(1) # Извежда: "One"
```

```
Math.calculate(3) # Извежда: "Other"
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.12. Lexical Scope

Лексикалният обхват се отнася до начина, по който анонимните функции достъпват променливи от външния си контекст (обхват). Анонимните функции в Elixir имат достъп до променливи, дефинирани в обхвата, в който са създадено анонимното им определение.

Пример:

```
x = 10
add_x = fn y -> x + y end
result = add_x.(5) # Извежда: 15
```

В примера `add_x` има достъп до `x`, който е дефиниран пред него.

Elixir използва лексикално обхождане, което означава, че обхватът на променливите е дефиниран от структурата на кода.

3.13. Named Functions

Наименуваните функции се дефинират в модули с ключовата дума `def`, а не чрез `defp` (частни функции). Те позволяват на потребителите на модула да ги извикват и да предават аргументи. Наименуваните функции са основният начин за деклариране на функции и често се използват при изграждането на API.

Пример:

```
defmodule MyMath do
  def add(a, b) do
    a + b
  end
end

result = MyMath.add(3, 5) # Извежда: 8
```


Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.14. Modules

Модулите в Elixir са основната единица за организация на кода. Те могат да съдържат функции, макроси и данни. Модулите се дефинират с ключовата дума `defmodule` и предлагат `Namespace` за организиране на функции.

Пример:

```
defmodule MyModule do
  def hello do
    "Hello, world!"
  end
end
```

Функции, дефинирани в модула `MyModule`, могат да се извикат с `MyModule.hello()`.

3.15. In-Line Definition

Функции могат да бъдат дефинирани `inline` в модул с `def`.

Вградена дефиниция на функции в Elixir е, когато функцията се определя директно в контекста, в който ще бъде извикана, напр. в другата функция. Можете да ги използвате, за да избегнете излишното разширяване на модулите.

Пример:

```
defmodule Example do
  def run do
    adder = fn a, b -> a + b end
    adder.(5, 7) # Извежда: 12
  end
end
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.16. Function Guards

Функционалните защиты добавят допълнителни условия към функциите.

Функционалните защиты са условни конструкции, които можете да използвате, за да контролирате коя имплементация на функцията да се извика, в зависимост от аргументите. Те се добавят след знака `when`.

Пример:

```
def test(value) when is_integer(value), do: "It's an integer"
```

Пример:

```
defmodule Math do
  def is_even(x) when rem(x, 2) == 0, do: true
  def is_even(_), do: false
end
```

```
Math.is_even(4) # Извежда: true
Math.is_even(5) # Извежда: false
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.17. Персонализирана защита

Можете да дефинирате персонализирани проверки в guard-и.

Персонализираната защита е част от функционалните защиты, но с по-специфично приложение. Можете да дефинирате свои собствени условия, които вашата функция трябва да проверява, преди да продължи с изпълнението. Обикновено се използва за валидиране на аргументи.

Пример:

```
defmodule Validator do
  def process_value(value) when is_integer(value) and value > 0 do
    "Valid value: #{value}"
  end
  def process_value(_), do: "Invalid value"
end
```

```
Validator.process_value(10) # Извежда: "Valid value: 10"
```

```
Validator.process_value(-5) # Извежда: "Invalid value"
```

Заклучение

Тези принципи относно анонимни функции, оператори, модулна структура, функционални защиты и други методи дават гъвкавост на разработчиците да организират и структурират своите приложения в Elixir. С познаване на тези концепции може да се изгради по-удобен и по-четим код.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.18. Multi-Clause Functions

Функции могат да имат множество клаузи, всяка от които може да отговаря на различни условия. Функциите с много клаузи в Elixir позволяват на една и съща функция да има различна логика в зависимост от аргументите. Това се постига чрез дефиниране на множество "клаузи" (или глави) на функцията, които се активират в зависимост от входящите данни.

Пример:

```
defmodule Math do
  def describe_number(0), do: "Zero"
  def describe_number(n) when n > 0, do: "Positive"
  def describe_number(n) when n < 0, do: "Negative"
end
```

```
Math.describe_number(0) # Извежда: "Zero"
Math.describe_number(10) # Извежда: "Positive"
Math.describe
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.19. Аргументи за съвпадение на шаблони

Arity и шаблони се използват за извикване на функции с различни аргументи.

Съвпадението на шаблони е основен механизъм в Elixir, който позволява на функцията да "съвпада" с входните аргументи в зависимост от техния вид или структура. Функции в Elixir могат да използват съвпадение на шаблони за идентифициране на аргументите, които да обработват.

Пример:

```
defmodule Shape do
  def area({:circle, radius}), do: :math.pi() * radius * radius
  def area({:rectangle, width, height}), do: width * height
end

Shape.area({:circle, 5})    # Извежда: 78.53981633974483
Shape.area({:rectangle, 4, 5}) # Извежда: 20
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Пример: Съвпадение на шаблони

```
_, [_ , {a}]] = ["Random string", [:an_atom, {24}]]
```

```
IO.puts(a)
```

Как работи?

Съвпадение на шаблони: В Elixir, когато използвате `_, [_ , {a}]]` вие казвате на интерпретатора каква структура очаквате от дясната страна на операцията (в този случай от списъка `["Random string", [:an_atom, {24}]]`).

Дясната страна: Структурата отдясно е:

```
["Random string", [:an_atom, {24}]]
```

Тук имаме списък с два елемента. Първият е "Random string" (стринг), а вторият е списък с два елемента: символа `:an_atom` и кортежа `{24}`.

Лявата страна: Вашият шаблон е `_, [_ , {a}]]`. Това означава:

Първият елемент (вляво) е игнориран, тъй като използвате `_`, който е "wildcard" (не се интересуваме от него).

Вторият елемент също е списък, в който:

Първият елемент (от списъка вляво) е игнориран (отново с `_`).

Вторият елемент е кортеж `{a}`, което означава, че стойността 24 ще бъде присвоена на променливата `a`.

Резултат: След успешното съвпадение, променливата `a` получава стойността 24, и след това `IO.puts(a)` извежда:
24

Символът `_` се използва в Elixir, когато не искате да запазвате стойността от дясната страна. Бъдете внимателни: не можете да използвате променливи, започващи с `_`, по-късно, тъй като те са окончателно игнорирани.



Пример: Използване на оператор "pin" (^). Сега да разгледаме примера с оператора "pin":

a = 25

b = 25

^a = b

Как работи този пример?

Присвояване на стойности:

Първо, присвояваме 25 на a. Сега a съдържа 25.

Присвояваме 25 на b, така че и b съдържа 25.

Оператор "pin" (^):

Операторът ^ се използва, за да се посочи, че искаме да "проверим" стойността на променливата a, а не да я променяме.

^a означава "вземи стойността на a (в случая 25)".

Съвпадение:

Сравнението ^a = b проверява дали стойността в b (която е 25) е равна на стойността на a (също 25).

Тъй като те са равни, операцията преминава без грешка.

3.20. Частни функции

Частните функции се дефинират с defp и не са достъпни извън модула.

Частните функции в Elixir се създават с ключовата дума defp. Те са достъпни само вътре в модула, в който са дефинирани, и не могат да бъдат извиквани извън него.

Пример:

```
defmodule Helper do
  def public_function do
    private_function() # Извикване на частна функция
  end

  defp private_function do
    "This is a private function"
  end
end

Helper.public_function() # Извежда: "This is a private function"
# Helper.private_function() # Обработка грешка: undefined function
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.21. Аргументи по подразбиране

Elixir не поддържа аргументи по подразбиране, но може да постигне подобно поведение чрез шаблони и guard-и. т.е. Elixir позволява задаването на аргументи по подразбиране в функции. Можете да дефинирате функции, при които определени аргументи имат стойности по подразбиране, за да спестите код и да осигурите по-оптимално извикване на функции.

Пример:

```
defmodule Greet do
  def hello(name \\ "World") do
    "Hello, #{name}!"
  end
end
```

```
Greet.hello()      # Извежда: "Hello, World!"
Greet.hello("Alice") # Извежда: "Hello, Alice!,,
```

3.22. Стенографията

Стенографията или "&" заместване е функция, която предоставя динамичност на кода в Elixir.

Операторът на улавяне (&) позволява създаването на анонимни функции по по-опростен начин, като използвате имената на аргументите, вместо да пишете целия fn ... end блок.

Пример:

```
add = &(&1 + &2)
result = add.(1, 2) # Извежда: 3
```


Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.23. Именуване на функции и Arity

Имената на функциите и тяхната арити (броя на аргументите) са важни за реда на извикванията и спецификациите им. Arity се отнася до броя на аргументите, които функцията приема. В Elixir имената на функциите и техният arity (броят на аргументите) формират уникален идентификатор. Можете да имате функции с едно и също име, но с различен arity.

Пример:

```
defmodule Math do
  def add(a, b), do: a + b
  def add(a, b, c), do: a + b + c
end
```

```
Math.add(1, 2)  # Извежда: 3
Math.add(1, 2, 3) # Извежда: 6
```

3.24. Функции и съвпадение на шаблони

Функциите могат да имат различни глави, които отговарят на различни шаблони. Функциите в Elixir могат да използват съвпадение на шаблони в своите параметри, което позволява лесно извличане на данни от сложни структури. Съвпадението на шаблони е основен механизъм за работа с данни в Elixir.

Пример:

```
defmodule TupleExtractor do
  def extract({first, second}), do: {second, first}
end
```

```
TupleExtractor.extract({1, 2}) # Извежда: {2, 1}
```

3.25. Частни функции

Частните функции не могат да бъдат извиквани извън модула и се дефинират с `defp`.

Както вече споменахме, частните функции (`defp`) са функция, която може да бъде извикана само в контекста на модула, в който е дефинирана. Това помага за инкапсулация и задава граници за видимост на функциите.

Пример:

```
defmodule MyModule do
  def public_function do
    private_function() # Може да извика частна функция
  end

  defp private_function do
    "I'm private"
  end
end
```

3.26. Защитници

Защитниците позволяват специфично поведение в зависимост от вида на аргументите.

Защитниците (`guards`) добавят допълнителни условия при определянето на коя функция да бъде извикана. Те могат да бъдат добавени след `when` ключовата дума и позволяват логическо условно изпълнение на функция.

Пример:

```
defmodule Math do
  def describe_number(x) when is_integer(x) and x > 0, do: "Positive integer"
  def describe_number(x) when is_integer(x) and x < 0, do: "Negative integer"
  def describe_number(_), do: "Not an integer"
end
```

```
Math.describe_number(10) # Извежда: "Positive integer"
Math.describe_number(-1) # Извежда: "Negative integer"
Math.describe_number("a") # Извежда: "Not an integer"
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

3.27. Аргументи по подразбиране

Няма преки аргументи по подразбиране, но можете да използвате шаблони и guard-и. Както вече отбелязахме, можете да добавяте аргументи по подразбиране към функции, което позволява на потребителите на функцията да не предоставят стойности за определени аргументи, ако те имат стойности по подразбиране.

Пример:

```
defmodule Greeter do
  def greet(name \\ "World") do
    "Hello, #{name}!"
  end
end
```

```
Greeter.greet()      # Извежда: "Hello, World!"
Greeter.greet("Alice") # Извежда: "Hello, Alice!"
```

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Използвана литература:

- [1] <https://coderlessons.com/tutorials/kompiuternoe-programmirovanie/vyuchit-eliksir/eliksir-operator>
- [2] <https://elixir-lang.readthedocs.io/en/latest/intro/3.html>
- [3] SAŠA JURIC, “Elixir in Action”, Second Edition, May 2015, ISBN 9781617292019
- [4] <https://gleam.run/cheatsheets/gleam-for-elixir-users/>
- [5] <https://devato.com/post/getting-started-with-elixir-functions>
- [6] <https://elixirschool.com/en/lessons/basics/modules/>
- [7] <https://elixir-examples.github.io/single-page>
- [8] <https://elixir-lang.org/getting-started/meta/quote-and-unquote.html>